



Assignment 3

Event-Driven Serverless Application for Review Analysis

Data-Intensive Computing

Tobias Ponesch (11818774)
Jakob Olsacher (01520348)
Alana Hoogmoed (11942398)
Ekene Edeh (12451120)
Paul Hübner (00371292)

July 1, 2026

1 Introduction

The increasing amount of user-generated content on digital platforms has created a demand for scalable and automated text analysis systems. Customer reviews provide valuable information about user experiences and product quality, but their volume makes manual analysis impractical. Cloud-native and serverless computing paradigms offer an effective solution by enabling scalable, event-driven processing pipelines.

This report presents the design and implementation of a serverless application for customer review analysis. Using AWS-compatible services provided through MiniStack, the application combines AWS Lambda functions, Amazon S3, DynamoDB, and AWS Systems Manager Parameter Store (SSM) to create an automated review processing workflow.

The application performs text preprocessing, profanity detection, sentiment analysis, and user moderation through a chain of event-driven functions. The primary goal of this project is to demonstrate the design and implementation of scalable serverless architecture rather than to optimize natural language processing performance.

2 Problem Overview

The objective of this project is to develop an event-driven serverless application that automatically processes customer reviews. The processing workflow is triggered whenever a new review is uploaded to an Amazon S3 bucket and subsequently executes a series of serverless functions.

For each review, the system analyzes the review text, summary, and rating information. The processing pipeline consists of three main tasks: text preprocessing, profanity detection, and sentiment analysis. In addition, the application maintains user-level statistics by counting the number of inappropriate reviews submitted by each customer. Users exceeding a predefined threshold of three impolite reviews are automatically marked as *banned*.

The system is implemented using multiple AWS Lambda functions connected through S3 object-creation events. DynamoDB stores user moderation state and per-review results, while bucket and table identifiers are retrieved through AWS Systems Manager Parameter Store. The main challenge of this project is therefore the design of a robust, scalable, and loosely coupled serverless architecture that coordinates multiple cloud services through event-driven interactions.

3 Methodology and Approach

The following section describes the design and implementation of the serverless review processing application. First, an overview of the event-driven architecture is provided, followed by a description of each processing step. Finally, the configuration management and testing approach are discussed.

3.1 Architecture Overview

The application uses three AWS Lambda functions in a MiniStack-hosted, event-driven pipeline. Amazon S3 bucket notifications connect the functions, while Amazon DynamoDB and AWS Systems Manager Parameter Store (SSM) provide state and configuration. Figure 1 illustrates the complete function chain, including the trigger events and interactions with the underlying cloud services.

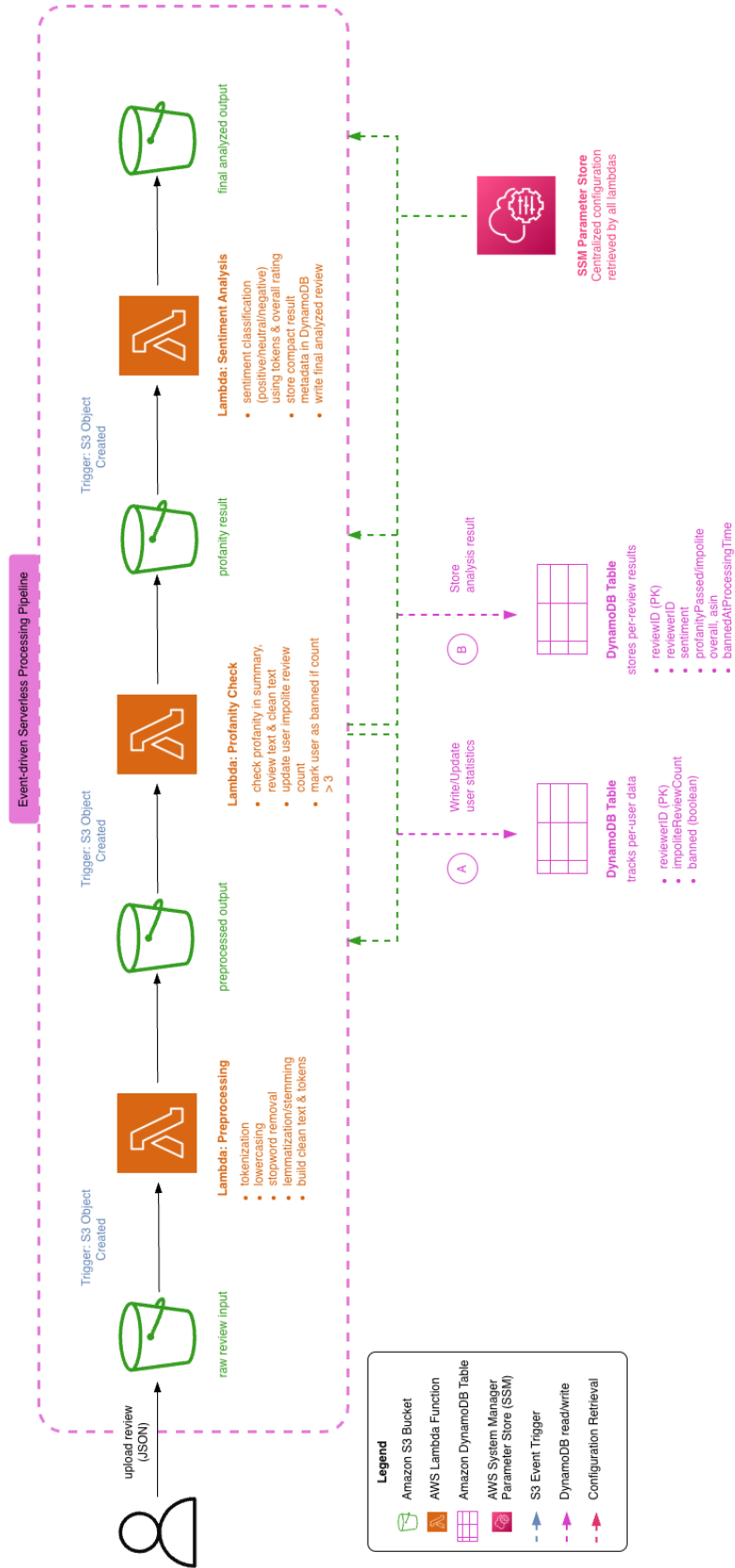


Figure 1: Overview of the implemented event-driven serverless review processing architecture

The workflow proceeds as follows:

1. A customer review in JSON format is uploaded to the raw review S3 bucket.
2. The upload triggers the preprocessing Lambda function, which performs tokenization, stop-word removal, and text normalization before storing the processed review in the preprocessed review bucket.
3. The creation of a new object in the preprocessed bucket triggers the profanity-check Lambda function. This function detects inappropriate language, updates user statistics in DynamoDB, and determines whether a user must be marked as banned.
4. The resulting review object is written to the profanity-checked review bucket, which subsequently triggers the sentiment-analysis Lambda function.
5. The sentiment-analysis function classifies the review sentiment, stores the resulting metadata in DynamoDB, and writes the final analyzed review to the analyzed review bucket.

3.2 Review Preprocessing

The preprocessing stage is implemented as an AWS Lambda function triggered by object creation events in the raw review bucket. For each incoming review, the summary and review texts are combined and subjected to several preprocessing operations.

First, the text is converted to lowercase to ensure case-insensitive processing. Tokenization is then performed using NLTK’s regular expression tokenizer when available, with a fallback regular expression implementation otherwise. Subsequently, common English stopwords are removed using either the NLTK stop-word corpus or a predefined fallback stop-word list. The implementation attempts WordNet lemmatization when the required NLTK resources are available, falls back to Snowball stemming when only the NLTK package is available, and otherwise applies deterministic suffix-based token normalization.

The preprocessing stage produces two outputs: a list of normalized tokens and a cleaned text representation. These outputs are included in a newly constructed payload containing the review fields required by the downstream stages.

3.3 Profanity Detection and User Moderation

The profanity-check Lambda function is triggered whenever a new review is inserted into the preprocessed review bucket. The stage analyzes the review summary, original review text, and normalized text representation for the presence of offensive language.

Profanity detection is primarily performed using the Python `profanityfilter` library¹, supplemented by a custom dictionary of additional inappropriate terms. If the external library is unavailable, the system automatically falls back to a dictionary-based approach.

In addition to identifying inappropriate reviews, this stage maintains user-level moderation statistics using Amazon DynamoDB. For each detected impolite review, the corresponding user’s violation counter is incremented. Users exceeding three inappropriate reviews are automatically marked as banned. The resulting moderation information is attached to the review object before it is forwarded to the next processing stage.

¹<https://pypi.org/project/profanityfilter/>

3.4 Sentiment Analysis and Result Storage

The sentiment-analysis Lambda function is triggered by object creation events in the profanity-checked review bucket. The sentiment classification combines lexical sentiment indicators extracted from the preprocessed tokens with the numerical product rating.

A predefined set of positive and negative terms is used to compute a lexical sentiment score. The score is then combined with the review rating to produce a final sentiment classification consisting of one of three classes: positive, neutral, or negative.

After classification, compact review metadata is stored in a DynamoDB results table. The stored information includes the review identifier, customer identifier, product identifier, sentiment classification, profanity status as well as user moderation status at processing time. Finally, the complete analyzed review object is written to the analyzed review bucket.

3.5 Configuration Management and Testing

To improve portability and maintainability, the S3 bucket names and DynamoDB table names are managed through AWS Systems Manager Parameter Store (SSM). Each Lambda function dynamically retrieves the resource identifiers it requires during execution.

The deployment script creates the MiniStack buckets, tables, SSM parameters, Lambda functions, and S3 event notifications. After this setup, the automated integration test uploads four deliberately impolite and negative reviews to the raw review bucket and waits for each review to reach the analyzed bucket. It verifies preprocessing output, profanity detection, negative sentiment classification, the impolite-review counter, automatic user banning after the fourth violation, and result storage in DynamoDB.

Separately, the full-devset runner clears previous MiniStack state, uploads each dataset review as an individual S3 object, waits for the pipeline to finish, and extracts aggregate statistics from DynamoDB. It uses a throttling strategy to keep MiniStack stable because uploading reviews to the raw S3 bucket was faster than MiniStack could process the resulting S3 events and Lambda invocations. Without throttling, the emulator accumulated a large backlog of pending events, which eventually pushed MiniStack into an overloaded state. The runner therefore compares the number of completed uploads with the number of reviews already written to the final analyzed review bucket. If this uploaded-but-not-yet-analyzed backlog exceeds a configured threshold, the runner pauses further uploads until the Lambda chain catches up. This keeps the local event queue bounded and made the complete devset run more reliable.

4 Results

The developed serverless application was evaluated using the provided `reviews_devset.json` dataset. The automated devset runner uploaded all reviews to the raw review bucket and verified the correct execution of the complete Lambda function chain. The final statistics were extracted from the DynamoDB result tables after successful processing.

Metric	Count
Processed reviews	78,829
Positive reviews	66,943
Neutral reviews	5,332
Negative reviews	6,554
Failed profanity check	3,684
Banned user	A320TMDV6KCFU

Table 1: Overview of review classification and moderation results for the complete `reviews_devset.json` run.

4.1 Discussion

The implementation successfully follows the required event-driven serverless design, with each processing stage triggered by the creation of an object in the corresponding S3 bucket. This keeps the Lambda functions loosely coupled because stages exchange S3 objects instead of invoking one another directly. The profanity-check and sentiment-analysis stages additionally interact with their respective DynamoDB tables, while the required bucket and table identifiers are retrieved from SSM Parameter Store.

A practical limitation was observed during the full devset evaluation in the local MiniStack environment. The upload rate of the devset runner was higher than the rate at which MiniStack could process S3 events and Lambda invocations. As a result, unthrottled uploads produced a large backlog of pending events and could overload the emulator. The throttling strategy in the runner addresses this by pausing uploads whenever the difference between uploaded and analyzed reviews becomes too large. This does not change the serverless pipeline itself, but it makes the local evaluation more stable and reproducible.

5 Conclusion

This project implemented an event-driven serverless application for automated customer review analysis using MiniStack, AWS Lambda, S3, DynamoDB, and SSM Parameter Store. The final architecture uses three S3-triggered Lambda functions for preprocessing, profanity detection, and sentiment analysis. User moderation state and compact result metadata are stored in DynamoDB, while SSM Parameter Store supplies the bucket and table identifiers required by the Lambda functions.

The complete devset run processed 78,829 reviews and produced the required sentiment, profanity, and banned-user statistics. The implementation therefore satisfies the main functional requirements of the assignment. The additional upload throttling strategy was important for stable local execution, since it prevented MiniStack from being overloaded by a backlog of S3 events during the full devset evaluation.